

1 SQL QUERIES FOR EMPLOYEE DATABASE WITH KEY CONSTRAINTS

AIM:

To create an employee database schema with key constraints using PL/SQL that maintains referential integrity between employees and departments. The program will create tables, apply primary and foreign key constraints, and insert sample records.

ALGORITHM:

1. Define the Departments table with dept_id as the primary key.
2. Define the Employees table with emp_id as the primary key and dept_id as a foreign key referencing Departments.
3. Create both tables in the database.
4. Insert sample records into Departments and Employees tables.
5. Commit the transactions to save changes permanently.

PROGRAM:

```
-- Create Departments table CREATE
TABLE Departments (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(50) NOT NULL
);

-- Create Employees table CREATE
TABLE Employees (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(100) NOT NULL,
    job_title VARCHAR2(100), dept_id
    NUMBER,
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);

-- Insert sample data into Departments
INSERT INTO Departments VALUES (1, 'HR');
INSERT INTO Departments VALUES (2, 'IT');
INSERT INTO Departments VALUES (3, 'Finance');
INSERT INTO Departments VALUES (4, 'Marketing'); INSERT
INTO Departments VALUES (5, 'Operations');

-- Insert sample data into Employees
INSERT INTO Employees VALUES (101, 'John Doe', 'HR Manager', 1);
INSERT INTO Employees VALUES (102, 'Jane Smith', 'Software Engineer', 2);
INSERT INTO Employees VALUES (103, 'Mike Brown', 'Accountant', 3); INSERT INTO
Employees VALUES (104, 'Emily Davis', 'Marketing
Analyst', 4);
INSERT INTO Employees VALUES (105, 'Robert Wilson', 'Operations Lead', 5);
INSERT INTO Employees VALUES (106, 'Lisa Taylor', 'IT Support', 2);

-- Commit changes
```

```
COMMIT;

SELECT * FROM Departments;
SELECT * FROM Employees;

-- Additional CRUD operations
-- CREATE: Insert a new department and employee INSERT
INTO Departments VALUES (6, 'Research');
INSERT INTO Employees VALUES (107, 'Alice Johnson', 'Research Scientist', 6);
COMMIT;

-- READ: Select employees working in the IT department (dept_id = 2) SELECT emp_id,
emp_name, job_title FROM Employees WHERE dept_id = 2;

-- UPDATE: Update job title of employee with emp_id = 102 UPDATE Employees
SET job_title = 'Senior Software Engineer' WHERE emp_id =
102;
COMMIT;

-- DELETE: Remove employee with emp_id = 105 (Robert Wilson) DELETE
FROM Employees WHERE emp_id = 105;
COMMIT;

-- READ: Select all employees after update and delete operations SELECT * FROM
Employees;

-- UPDATE: Change department name of dept_id = 4 to 'Digital Marketing'
UPDATE Departments
SET dept_name = 'Digital Marketing' WHERE
dept_id = 4;
COMMIT;

-- DELETE: Remove the 'Research' department and any employees assigned to it
DELETE FROM Employees WHERE dept_id = 6;
DELETE FROM Departments WHERE dept_id = 6;
```

```
COMMIT;  
  
-- READ: Final state of Departments and Employees  
SELECT *  
FROM Departments;  
SELECT * FROM Employees;
```

RESULT:

Thus, The script has been created with two related tables with sample records that enforce key constraints and maintain relational integrity between employees and their departments.

2 IMPLEMENT DCL, TCL COMMANDS

AIM:

To implement Data Control Language (DCL) and Transaction Control Language (TCL) commands in a university database context. This program will demonstrate granting and revoking privileges, as well as managing transactions using commit, rollback, and savepoint.

ALGORITHM:

1. Grant specific privileges (SELECT, INSERT, UPDATE, DELETE) on university tables to users/roles.
2. Revoke some privileges to demonstrate control.
3. Use TCL commands to manage transactions, including COMMIT, ROLLBACK, and SAVEPOINT.
4. Show sample operations illustrating transaction control and privilege management

PROGRAM:

```
-- Create Departments table CREATE
TABLE Departments (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(50) NOT NULL
);

-- Insert sample department records
INSERT INTO Departments VALUES (1, 'Computer Science'); INSERT
INTO Departments VALUES (2, 'Mathematics');
COMMIT;

-- Create Students table
CREATE TABLE Students (
    student_id NUMBER PRIMARY KEY,
    name VARCHAR2(100) NOT NULL,
    dob DATE,
    dept_id NUMBER,
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);
COMMIT;-----

-----

-- DCL Commands: Self-Grant & Revoke Privileges
```

```
-- Grant privileges to yourself (the current user) GRANT SELECT,  
INSERT ON Students TO USER;
```

```
-- Demonstrate revoke  
REVOKE INSERT ON Students FROM USER;
```

```
-- You can check what privileges you hold SELECT  
TABLE_NAME, PRIVILEGE, GRANTEE  
FROM USER_TAB_PRIVS  
WHERE TABLE_NAME = 'STUDENTS';
```

```
-----  
  
-- TCL Commands: Transaction Control  
-----
```

```
BEGIN
```

```
-- Insert a new student
```

```
INSERT INTO Students (student_id, name, dob, dept_id)  
VALUES (304, 'David Tennant', TO_DATE('2002-12-20', 'YYYY-MM-DD'), 1);
```

```
-- Savepoint
```

```
SAVEPOINT sp1;
```

```
-- Insert another student
```

```
INSERT INTO Students (student_id, name, dob, dept_id)  
VALUES (305, 'Emma Watson', TO_DATE('2001-04-15', 'YYYY-MM-DD'), 2);
```

```
-- Rollback to savepoint
```

```
ROLLBACK TO sp1;
```

```
-- Commit transaction (David Tennant remains) COMMIT;
```

```
END;
```

```
/
```

```
-- Check Students table after TCL operations
```

```
SELECT * FROM Students WHERE student_id IN (304, 305);
```

OUTPUT:

student_id	name	dob	dept_id
304	David Tennant	2002-12-20	1

RESULT:

Thus, this output displays students with their enrolled courses for each semester, demonstrating a successful join operation among Students, Enrollments, and Courses tables.

3 IMPLEMENT FIRST, SECOND AND THIRD NORMALIZATION TECHNIQUES

AIM:

To demonstrate the implementation of First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) by creating a fresh table, identifying anomalies, and then decomposing it step-by-step into normalized tables.

ALGORITHM:

1. Create an initial unnormalized table **Student_Courses** with repeating groups and partial dependencies.
2. Apply 1NF to remove repeating groups and ensure atomic column values.
3. Apply 2NF by removing partial dependencies (splitting tables where non-key attributes depend on part of a composite key).
4. Apply 3NF by removing transitive dependencies (splitting tables so that non-key attributes depend only on the whole key).
5. Insert sample data in each stage to illustrate normalization.

PROGRAM:

```
CREATE TABLE Student_Courses ( student_id
    NUMBER,
    student_name VARCHAR2(100),
    course_id    NUMBER,
    course_name VARCHAR2(100),
    instructor VARCHAR2(100),
    instructor_phone VARCHAR2(15),
    PRIMARY KEY (student_id, course_id)
);
```

```
INSERT INTO Student_Courses VALUES (1, 'Alice Smith', 101, 'Database Systems',
'Dr. Alan Turing', '1234567890');
INSERT INTO Student_Courses VALUES (1, 'Alice Smith', 102, 'Algorithms', 'Dr. Ada
Lovelace', '1234567891');
INSERT INTO Student_Courses VALUES (2, 'Bob Johnson', 101, 'Database Systems',
```

```
CREATE TABLE Students (  
    student_id NUMBER PRIMARY KEY,  
    student_name VARCHAR2(100)  
);
```

```
CREATE TABLE Courses (  
    course_id NUMBER PRIMARY KEY,  
    course_name VARCHAR2(100),  
    instructor VARCHAR2(100),  
    instructor_phone VARCHAR2(15)  
);
```

```
CREATE TABLE Enrollments  
    ( student_id NUMBER,  
    course_id NUMBER,  
    PRIMARY KEY (student_id, course_id),  
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES  
Students(student_id),  
    CONSTRAINT fk_course FOREIGN KEY (course_id) REFERENCES  
Courses(course_id)  
);
```

Insert sample normalized data:

```
INSERT INTO Students VALUES (1, 'Alice Smith'); INSERT INTO
```

```
Turing', '1234567890');
INSERT INTO Courses VALUES (102, 'Algorithms', 'Dr. Ada Lovelace', '1234567891');

INSERT INTO Enrollments VALUES (1, 101);
INSERT INTO Enrollments VALUES (1, 102);
INSERT INTO Enrollments VALUES (2, 101);

COMMIT;
```

```
CREATE TABLE Instructors (
    instructor_name VARCHAR2(100) PRIMARY KEY,
    instructor_phone VARCHAR2(15)
);

ALTER TABLE Courses DROP COLUMN instructor_phone;

ALTER TABLE Courses ADD CONSTRAINT fk_instructor FOREIGN KEY (instructor)
REFERENCES Instructors(instructor_name);

Insert instructor data:
INSERT INTO Instructors VALUES ('Dr. Alan Turing', '1234567890'); INSERT INTO
Instructors VALUES ('Dr. Ada Lovelace', '1234567891');

COMMIT;
SELECT * FROM Students
SELECT * FROM Courses
```

OUTPUT:

student_id	student_name
1	Alice Smith
2	Bob Johnson

instructor_name	instructor_phone
Dr. Alan Turing	1234567890
Dr. Ada Lovelace	1234567891

course_id	course_name	instructor
101	Database Systems	Dr. Alan Turing
102	Algorithms	Dr. Ada Lovelace

student_id	course_id
1	101
1	102
2	101

RESULT:

This program fully shows the process of applying 1NF, 2NF, and 3NF to a practical university enrollment schema with sample data, reducing redundancy and ensuring data integrity.

4 IMPLEMENT FUNCTIONS/PROCEDURES TO BEGIN, COMMIT AND ROLLBACK TRANSACTIONS

AIM:

To implement PL/SQL functions and procedures that manage transactions explicitly by beginning, committing, and rolling back transactions. These will demonstrate programmatic control over transaction boundaries.

ALGORITHM:

1. Create a procedure to start a transaction (conceptually, PL/SQL starts implicit transactions automatically, so this can be a placeholder).
2. Create a procedure for committing the current transaction.
3. Create a procedure for rolling back the current transaction.
4. Write a simple PL/SQL block that uses these procedures while performing insert/update operations to demonstrate transaction control.
5. Use exception handling to conditionally commit or rollback.

PROGRAM:

```
CREATE TABLE test_table ( id  
    NUMBER PRIMARY KEY,  
    data VARCHAR2(100)  
);
```

```
-- Procedure to begin transaction (placeholder as transactions begin implicitly)
CREATE OR REPLACE PROCEDURE Begin_Transaction IS BEGIN
    DBMS_OUTPUT.PUT_LINE('Transaction begun (implicit in PL/SQL)'); END;
/
```

```
-- Procedure to commit transaction
CREATE OR REPLACE PROCEDURE Commit_Transaction IS
BEGIN
```

```

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transaction committed'); END;
/

-- Procedure to rollback transaction
CREATE OR REPLACE PROCEDURE Rollback_Transaction IS
BEGIN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Transaction rolled back'); END;
/

-- Demonstration block using above procedures DECLARE
v_error_occurred BOOLEAN := FALSE; BEGIN
Begin_Transaction;

-- Sample insert or update (create a test table if needed) BEGIN
INSERT INTO test_table(id, data) VALUES (1, 'Sample Data'); EXCEPTION
WHEN OTHERS THEN
    v_error_occurred := TRUE;
    DBMS_OUTPUT.PUT_LINE('Error during insert: ' || SQLERRM); END;

IF v_error_occurred THEN
    Rollback_Transaction;
ELSE
    Commit_Transaction; END
IF;
END;
/

```

OUTPUT:

Transaction begun

(implicit in PL/SQL)

Transaction committed

RESULT:

Thus, This program shows controlled execution of transactions where changes are only saved on success, and any failure triggers rollback to ensure data integrity, illustrating key practices for reliable database programming.